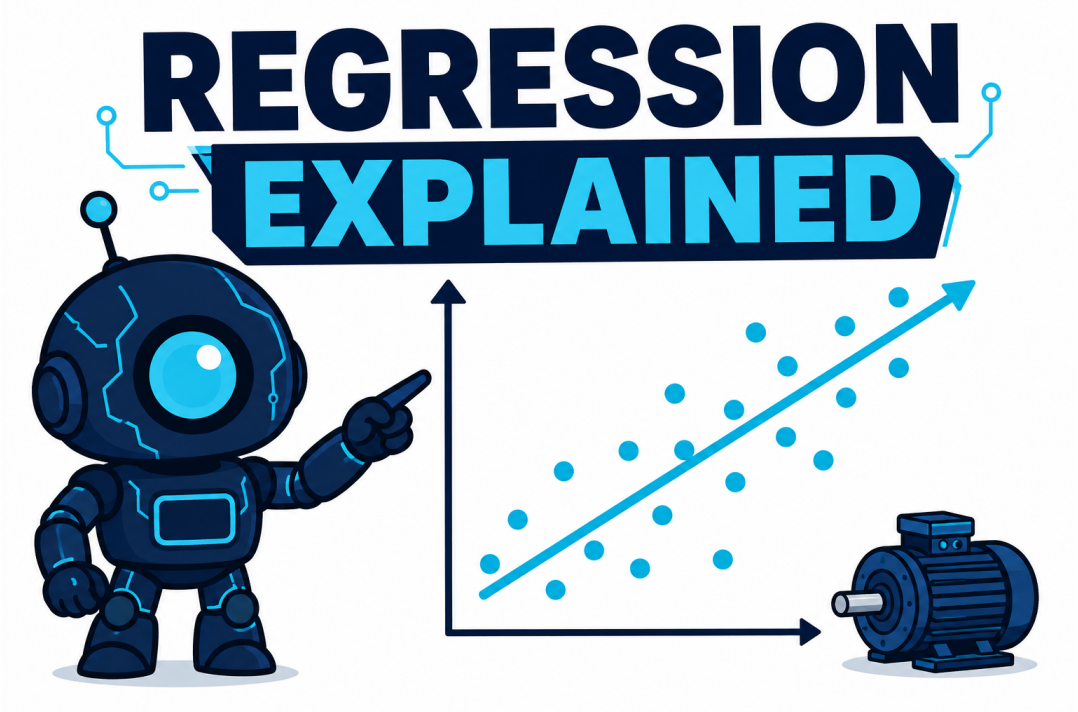
AI MECH LAB

AI Course — Beginner Track



Module 4, Topic 4.2

Regression

Lesson

You already know the big fork in the road: supervised learning predicts a known kind of outcome from labeled data. Now you zoom into the first branch of that fork. Regression is supervised learning where the thing you're predicting is a number — not a category, not a yes/no, a number that can fall anywhere on a continuous scale. If your target is "how much," "how many," or "how far," you're doing regression.

What makes a target a regression problem

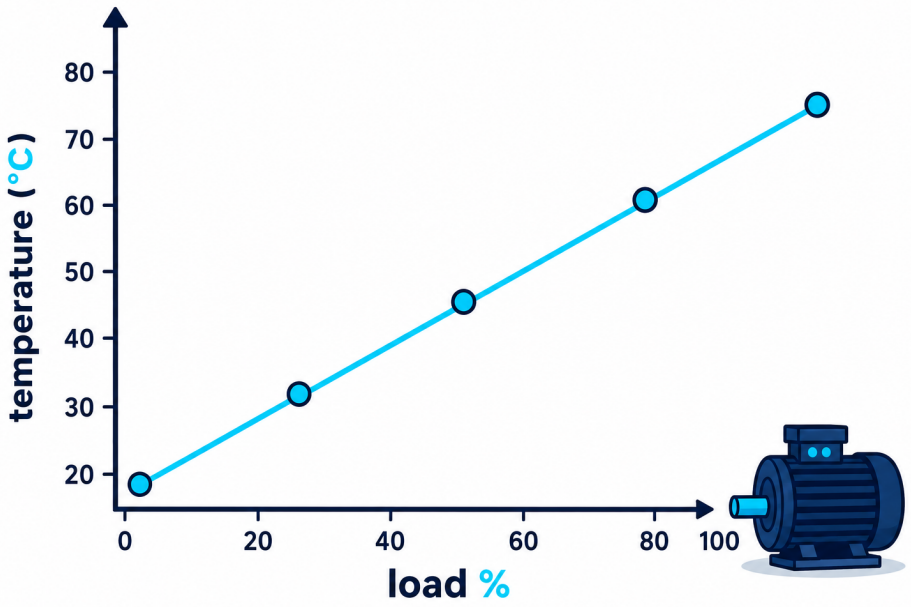
The test is simple: can the answer be any value along a range, including values in between? Predicting how many watts a solar panel will output given the current irradiance and panel temperature is regression — 187.4 watts is a valid answer, and so is 187.5. Predicting how many liters of fuel a diesel generator will burn per hour at a given load is regression. Predicting how many hours a car battery will hold charge before it needs recharging is regression. In every case, the label isn't one of a fixed set of categories — it's a number on a continuous scale.

Compare that to something like "is this generator overheating or not" — that has exactly two possible answers, nothing in between. That's not regression; you'll see that kind of problem in the next topic.

The simplest possible regression: a straight line

Strip regression down to its core, and the simplest version is fitting a straight line through your data. Say you're tracking one feature — motor load, as a percentage — against one label — motor temperature in Celsius. Plot load on the x-axis and temperature on the y-axis, and if there's a real relationship, the points will roughly trend upward together: more load, more heat.

```
load_percent = [20, 40, 60, 80, 100]
temperature_c = [35, 42, 51, 58, 67]
```

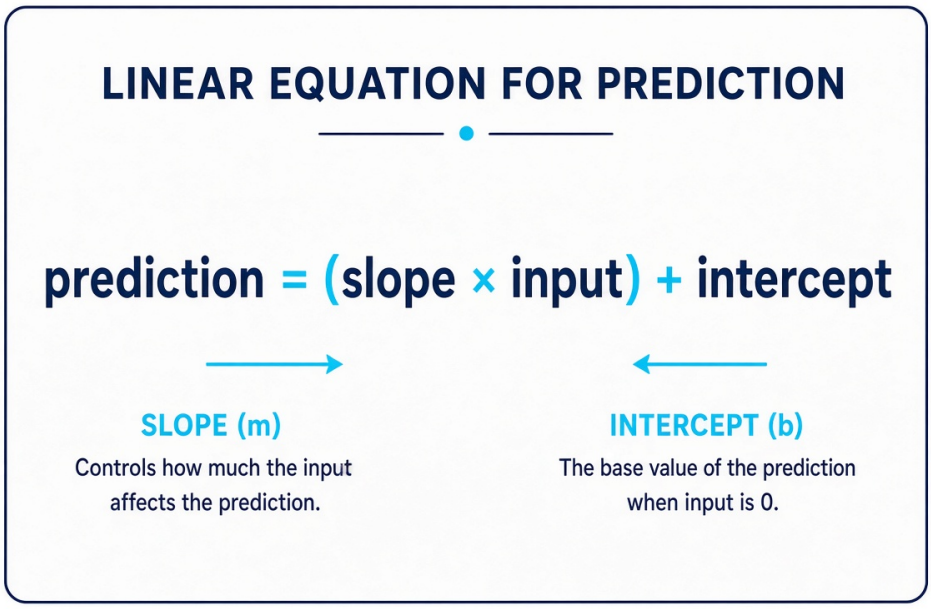


Motor load vs. temperature, with the best-fit line drawn through the points.

A linear regression model's job is to find the single straight line that best fits through those points — close to as many of them as possible, on average. Once it has that line, predicting a new value is just reading a point off the line: give it a load percentage it hasn't seen, like 70%, and it tells you the temperature the line predicts at that point.

The line has a formula

```
prediction = (slope * input) + intercept
```

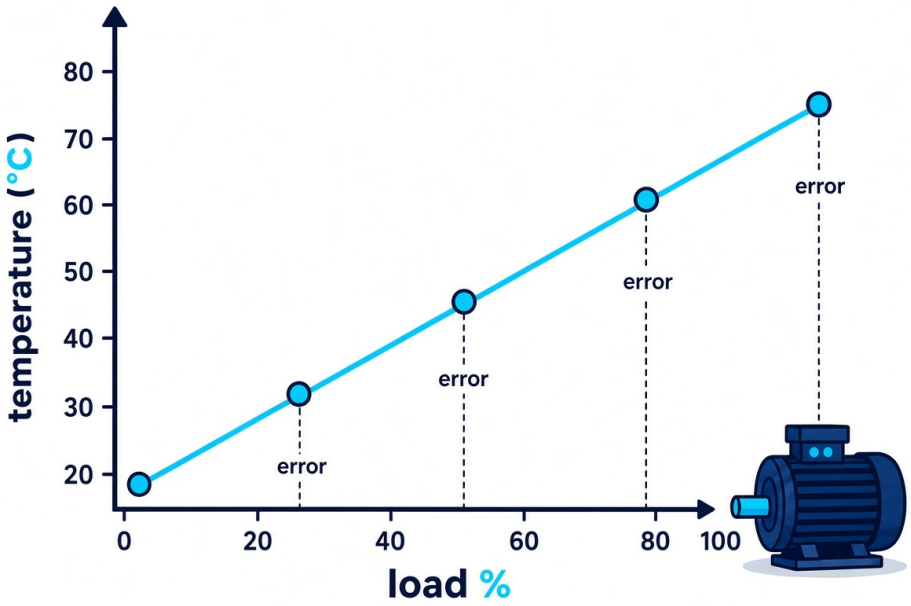


Slope controls sensitivity to input; intercept is the baseline at zero.

The slope tells you how much the prediction changes for every one-unit increase in the input — in the motor example, how many extra degrees each percentage point of added load costs you. The intercept is where the line sits when the input is zero — roughly the motor's baseline temperature at no load. Training a regression model is the process of the algorithm searching for the slope and intercept values that make the line fit the actual data as closely as possible — it's not guessing at a shape, it's calculating the two numbers that define that shape.

Real data doesn't sit perfectly on a line

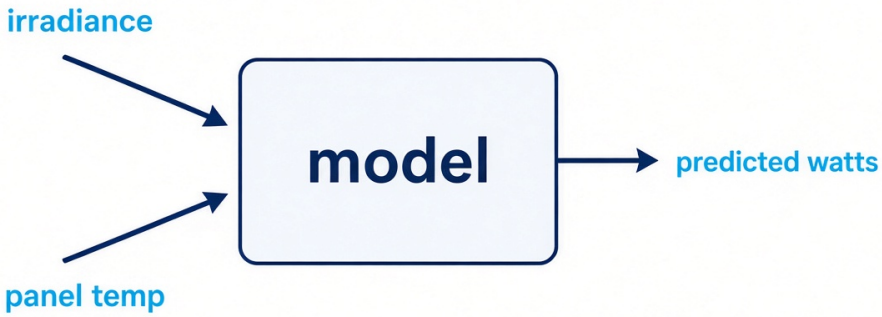
In practice, your points will scatter a bit around the line rather than sitting exactly on it — sensor noise, ambient temperature on the day of the reading, airflow around the motor, factors you didn't capture. That's expected. A good regression model doesn't need every point to sit exactly on the line; it needs the line to be the best overall fit, minimizing how far off it is across all the points combined. The vertical distance between an actual reading and the line at that spot is called the error, or residual, and a big part of how these models are trained is by working to minimize that error across the whole dataset.



The vertical gap between each point and the line is the error being minimized.

Regression with more than one feature

Real problems rarely use just one input. Take a solar setup: panel output in watts depends on more than one thing at once — solar irradiance (how strong the sunlight is) and panel temperature (panels actually lose some efficiency as they heat up). Feed both features in together, and the model finds a weight for irradiance and a weight for panel temperature, combining them into a single predicted wattage.

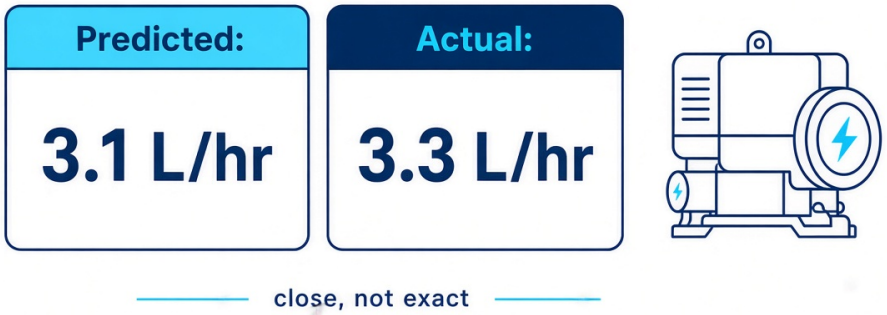


Two features combining into one predicted output.

That's still regression, just with two inputs feeding one prediction instead of one. Add a third feature — say, panel tilt angle — and the idea doesn't change, only the number of weights being learned.

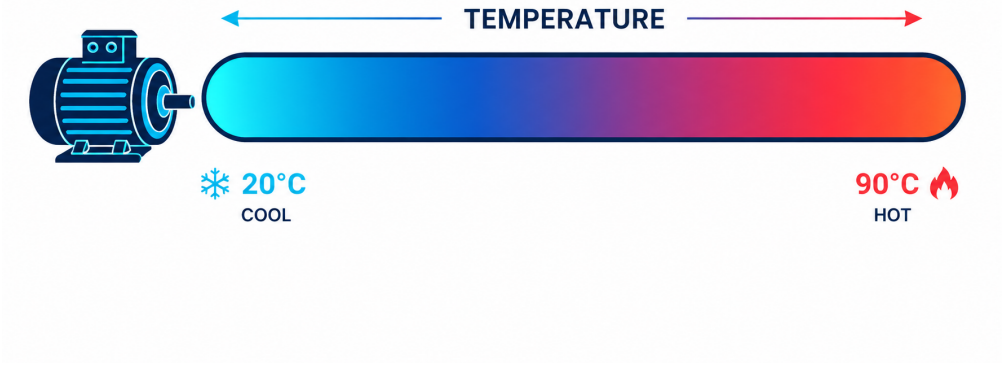
Measuring how good a regression model actually is

Because the answer is a number and not a category, "correct" isn't a clean yes/no the way it might be elsewhere — a predicted fuel burn of 3.1 liters/hour against an actual of 3.3 isn't wrong, it's just off by 0.2. Regression models are judged by how far off their predictions are, on average, across the whole dataset.



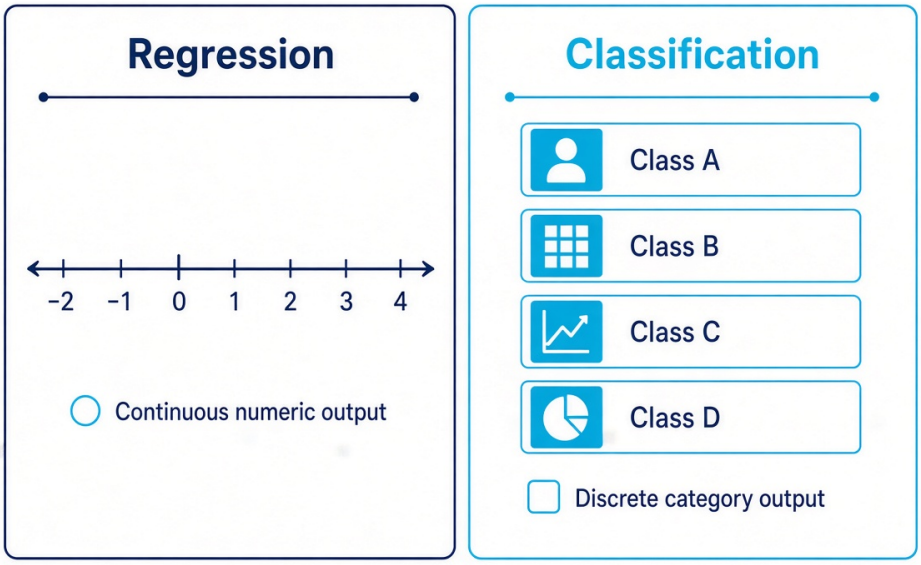
A close prediction isn't a wrong one — it's judged by distance from the truth.

Smaller average error means a better-fitting model. You'll get hands-on with the specific ways this gets measured once you're building models directly — for now, the concept to hold onto is: regression is graded by distance from the truth, not by exact match.



A continuous scale has no fixed steps — any value in between is valid.

Where regression fits next to classification

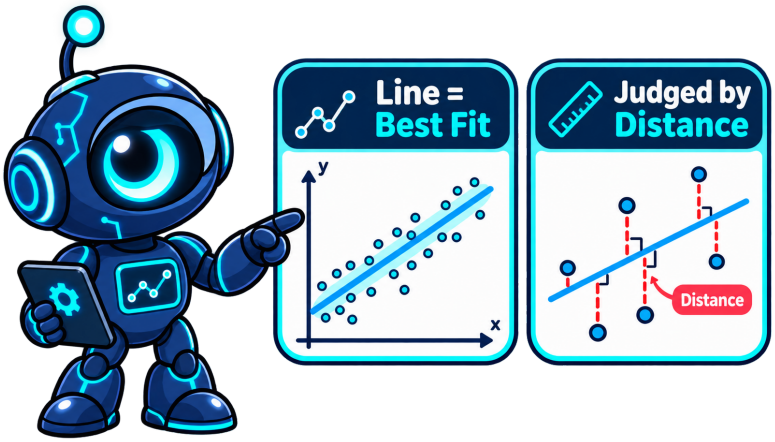


Regression outputs a continuous number; classification outputs a discrete category.

That contrast is exactly where you're headed next: Classification, the other branch of supervised learning, where the target isn't a number at all — it's a category.

Where this leaves you

By the end of this lesson, you should be able to recognize a regression problem on sight — the target is a number on a continuous scale, whether that's temperature, fuel burn, wattage, or runtime — and understand what a trained regression model is actually doing underneath: finding the combination of weights that turns input features into a number as close as possible to the real answer, judged by how far off it lands rather than whether it's exactly right. Next, you flip to the other branch of supervised learning — Classification — where the target isn't a number at all, but a category.



The two core ideas: line as best fit, judged by distance.